Parameterized Modules and Local Parameters

October 12, 2026

Contents

1 Introduction **2**

2 Module-Level Parameter Using `#(...)` **2**
2.1 Design Module . . . 2
2.2 Explanation . . . 2

3 Complete Testbench for the Parameterized Module **3**

4 Parameter Overriding **4**
4.1 Default Parameter . . . 4
4.2 Overridden Parameter . . . 4

5 Local Parameter Using `localparam` **4**
5.1 Definition . . . 4
5.2 Syntax . . . 5
5.3 Illustrative Example: Parameterized Counter . . . 5
5.4 Explanation . . . 6
5.5 Counter Instantiation . . . 6
5.6 Attempting to Override a Local Parameter . . . 7

6 Parameter Declared Inside the Module Body **7**

7 Comparison: Parameter vs. Localparam **8**
7.1 Simple Example . . . 9

8 Expected Simulation Output **9**

9 Simulation Using Icarus Verilog **9**

10 Key Points to Remember **10**

11 Conclusion **10**

1 Introduction

In Verilog HDL, a `parameter` defines a constant whose value can be specified when a module is instantiated.

A `localparam` also defines a constant, but its value cannot be directly overridden through the module instantiation parameter override mechanism.

Parameterized modules make hardware designs reusable. For example, the same module can be used to create 8-bit, 16-bit, or 32-bit circuits without rewriting the module.

The main concepts covered in this document are:

- Module parameter declared using `#(...)`.
- Parameter declared inside the module body.
- Local parameter declared using `localparam`.
- Parameter overriding and module instantiation.
- Simulation and expected output.

2 Module-Level Parameter Using `#(...)`

2.1 Design Module

```
module my_module #(
    parameter N = 8
) (
    input  wire [N-1:0] a,
    output wire [N-1:0] y
);

    assign y = a;

endmodule
```

2.2 Explanation

- `parameter N = 8`: Defines the default parameter value as 8.
- `[N-1:0]`: Defines an N-bit signal. When N = 8, the bit range is [7:0].
- `assign y = a`: Continuously copies the input value to the output.
- `#(...)`: Declares the module parameter port list, allowing parameter values to be overridden during instantiation.

3 Complete Testbench for the Parameterized Module

The testbench instantiates the module twice: once with the default width of 8 bits and once with an overridden width of 16 bits.

```
`timescale 1ns/1ps

module tb_my_module;

    reg  [7:0]  a8;
    wire [7:0]  y8;

    reg  [15:0] a16;
    wire [15:0] y16;

    // Instance 1: Default N = 8
    my_module u1 (
        .a(a8),
        .y(y8)
    );

    // Instance 2: Override N = 16
    my_module #(
        .N(16)
    ) u2 (
        .a(a16),
        .y(y16)
    );

    initial begin
        $dumpfile("my_module.vcd");
        $dumpvars(0, tb_my_module);

        a8  = 8'hA5;
        a16 = 16'hABCD;

        #10;

        $display("8-bit  : a = %h, y = %h",
                 a8, y8);

        $display("16-bit : a = %h, y = %h",
                 a16, y16);

        #10;
        $finish;
    end

endmodule
```

4 Parameter Overriding

A parameter can be overridden when a module is instantiated.

4.1 Default Parameter

```
my_module u1 (
    .a(a8),
    .y(y8)
);
```

Since no parameter override is specified, the default value N = 8 is used.

Therefore,

$$\text{Width of } a = 8 \text{ bits}$$

$$\text{Width of } y = 8 \text{ bits}$$

4.2 Overridden Parameter

```
my_module #(
    .N(16)
) u2 (
    .a(a16),
    .y(y16)
);
```

Here, the parameter N is overridden to 16.

Therefore,

$$\text{Width of } a = 16 \text{ bits}$$

$$\text{Width of } y = 16 \text{ bits}$$

Important: Each instance can have its own parameter value. The original module definition remains unchanged.

5 Local Parameter Using `localparam`

5.1 Definition

A `localparam` defines a constant whose value cannot be directly overridden through a module instance's parameter override.

It is commonly used for internal constants and values derived from other parameters.

5.2 Syntax

```
localparam CONSTANT_NAME = VALUE;
```

For example:

```
parameter WIDTH = 8;
localparam HALF_WIDTH = WIDTH / 2;
```

Here, `WIDTH` is configurable, whereas `HALF_WIDTH` is calculated internally.

If `WIDTH = 8`, then:

$$HALF_WIDTH = \frac{8}{2} = 4$$

If `WIDTH = 16`, then:

$$HALF_WIDTH = \frac{16}{2} = 8$$

The local parameter changes its calculated value when its dependency changes, but it cannot itself be directly overridden by the module instantiator.

5.3 Illustrative Example: Parameterized Counter

Consider a counter whose width can be configured. The maximum count value is calculated internally using a local parameter.

```
module counter #(
    parameter WIDTH = 4
) (
    input  wire clk,
    input  wire rst,
    output reg  [WIDTH-1:0] count
);

    localparam MAX_COUNT = (2**WIDTH) - 1;

    always @(posedge clk) begin
        if (rst)
            count <= 0;
        else if (count == MAX_COUNT)
            count <= 0;
        else
            count <= count + 1'b1;
    end

endmodule
```

5.4 Explanation

- `WIDTH` determines the number of counter bits.
- `MAX_COUNT` is calculated using `WIDTH`.
- `localparam` prevents direct external overriding of `MAX_COUNT`.
- The counter resets to zero when the maximum count is reached.

For a 4-bit counter:

$$MAX_COUNT = 2^4 - 1 = 15$$

The counter cycles through 0, 1, 2, ..., 15, and then returns to 0.

For an 8-bit counter:

$$MAX_COUNT = 2^8 - 1 = 255$$

The counter cycles through 0 to 255 and then returns to 0.

Note: The expression above is illustrative. For very large widths, use appropriately sized arithmetic expressions to avoid expression-width limitations.

5.5 Counter Instantiation

```
module tb_counter;

    reg clk;
    reg rst;
    wire [7:0] count;

    // WIDTH is overridden to 8.
    counter #(
        .WIDTH(8)
    ) u_counter (
        .clk(clk),
        .rst(rst),
        .count(count)
    );

    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end

    initial begin
        rst = 1;
        #12;
        rst = 0;

```

```
        #300;
        $finish;
    end

endmodule
```

In this testbench, the counter has an 8-bit output. Its local parameter is calculated from the selected width:

$$MAX_COUNT = 2^8 - 1 = 255$$

The testbench does not need to specify `MAX_COUNT`.

5.6 Attempting to Override a Local Parameter

The following instantiation is invalid because `MAX_COUNT` is declared as a local parameter:

```
counter #(
    .WIDTH(4),
    .MAX_COUNT(10)
) u_counter (
    .clk(clk),
    .rst(rst),
    .count(count)
);
```

The correct instantiation is:

```
counter #(
    .WIDTH(4)
) u_counter (
    .clk(clk),
    .rst(rst),
    .count(count)
);
```

With this configuration, `MAX_COUNT` is calculated internally as 15.

6 Parameter Declared Inside the Module Body

A parameter can also be declared inside the module body. It remains an overridable parameter.

```
module counter_body (
    input  wire clk,
    input  wire rst,
    output reg  [3:0] count
);

    parameter MAX_COUNT = 9;
```

```
8
9       always @(posedge clk) begin
10          if (rst)
11              count <= 0;
12          else if (count == MAX_COUNT)
13              count <= 0;
14          else
15              count <= count + 1'b1;
16      end
17
18  endmodule
```

The parameter can generally be overridden during instantiation:

```
counter_body #(
    .MAX_COUNT(5)
) u_counter (
    .clk(clk),
    .rst(rst),
    .count(count)
);
```

This counter cycles through 0 to 5.

A body-declared `parameter` is not the same as a `localparam`. The former can be overridden; the latter cannot be directly overridden.

7 Comparison: Parameter vs. Localparam

Feature	**parameter**	**localparam**
Defines a constant	Yes	Yes
Can have a default value	Yes	Yes
Directly overridable at instantiation	Yes	No
Can depend on another parameter	Yes	Yes
Typical purpose	External configuration	Internal constants

7.1 Simple Example

```
module example #(
    parameter WIDTH = 8
) (
    input wire clk
);

    localparam HALF_WIDTH = WIDTH / 2;

endmodule
```

For `WIDTH = 8`, `HALF_WIDTH = 4`.

For `WIDTH = 16`, `HALF_WIDTH = 8`.

The local parameter is recalculated during elaboration after the configurable parameter value is selected.

8 Expected Simulation Output

For the original `my_module` testbench, the expected output is:

```
8-bit  : a = a5, y = a5
16-bit : a = abcd, y = abcd
```

The output matches the input because the module uses:

```
assign y = a;
```

The counter example has a different purpose: it demonstrates how a local parameter calculates an internal limit. Its output changes over time as clock edges occur.

9 Simulation Using Icarus Verilog

Save the parameterized design and its testbench in separate files:

- `my_module.v`
- `tb_my_module.v`

Compile and simulate:

```
iverilog -g2012 -o sim my_module.v tb_my_module.v
vvp sim
```

For the counter example, save the module and testbench in separate files named `counter.v` and `tb_counter.v`, then run:

```
iverilog -g2012 -o sim_counter counter.v tb_counter.v
vvp sim_counter
```

The counter testbench above does not include display statements, so it will not print counter values to the terminal unless `$display` or `$monitor` is added.

10 Key Points to Remember

- `parameter` defines a configurable constant.
- `module #(parameter ...)` declares parameters in the module header.
- A parameter declared inside the module body is also generally overridable.
- `localparam` defines an internally fixed constant that cannot be directly overridden.
- A local parameter can depend on a regular parameter.
- Parameter values are resolved during elaboration, before normal simulation begins.
- Parameters are not runtime variables or registers.

11 Conclusion

Use `parameter` when a module needs configurable features such as data width, FIFO depth, or a selectable operating limit.

Use `localparam` when a constant should be calculated or maintained internally, such as a derived maximum count, address width, or state encoding.

> **Rule to remember:**
>
> `parameter` = configurable constant.
>
> `localparam` = internal constant that cannot be directly overridden at module instantiation.